RAPPORT TECHNIQUE · TOKEN-ECO

Ce que token-eco économise vraiment

Six séries de mesures sur un plugin Claude Code qui réduit les tokens lus et écrits : qualité des outils, coût en conditions contrôlées, adoption en usage réel et sessions multi-agents.

Versions v0.3 → v0.7.4 | 22 – 28 octobre 2026 | Sonnet 5.5 · Opus 5.5 | Claude Code 2.1.29x – 2.1.31x | Données : server/bench/

−48 %
coût d'une tâche longue multi-agents (Sonnet, meilleure variante)

0 → 93 %
part des lectures d'un relecteur passant par token-eco une fois équipé

4–24 %
taille d'une lecture ciblée vs fichier entier, fidélité 92–100 %

63 / 63
exécutions réussies au banc d'adoption, aucune perte de qualité

RÉSUMÉ

token-eco fournit à Claude Code des outils de lecture, recherche et édition de fichiers conçus pour renvoyer moins de tokens, un agent principal qui les impose, une consigne injectée aux sous-agents et un hook qui compacte les sorties Bash et redirige les lectures de gros fichiers. **(1)** Hors ligne, les outils renvoient 4 à 24 % de la taille d'une lecture complète avec une fidélité structurelle de 92 à 100 % (TS, JS, Python) ; la recherche compacte donne les mêmes résultats que `grep` pour 47 % de ses tokens. **(2)** En A/B mono-agent sur des tâches courtes (Sonnet), le coût baisse de 8 à 10 % sans perte de réussite (56/56). **(3)** Sur des sessions réelles scénarisées, aucune économie en dollars n'est démontrée avec Sonnet (−7 % à +22 % selon les runs) ; un run Opus donne −13 %. **(4)** En usage réel (30 jours), les outils n'étaient presque pas utilisés (4,7 % du fil principal, 1,2 % des sous-agents) : un autre plugin imposait son agent et des agents de projet n'y avaient pas accès. **(5)** Dans un banc multi-agents reproduisant cette situation (63 exécutions, toutes réussies), équiper le relecteur fait passer sa couverture de 0 % à 39–93 % ; sur des tâches courtes le coût ne change pas de façon mesurable, tandis que sur une tâche longue les variantes token-eco coûtent **19 à 48 % de moins**, avec séparation complète des valeurs dans deux cas sur quatre (p = 0,10, minimum atteignable avec 3 essais). Une partie de ce gain vient de l'agent principal (moins de tours

d'orchestration). Aucun résultat n'atteint le seuil de 5 % : ce sont des tendances cohérentes, pas des preuves.

6	≈ 62 $	10
séries de mesures, de la qualité hors ligne à l'usage réel	consommés par le banc d'adoption (prix équivalent API)	conclusions, chacune avec son niveau de preuve

01 Contexte et questions

Dans Claude Code, chaque appel d'outil est un tour complet : le modèle relit tout le contexte (depuis le cache de prompt), puis le résultat de l'outil est ajouté au contexte et relu à chaque tour suivant. Le coût d'une session dépend donc de la taille des résultats, du nombre de tours et de la taille du contexte accumulé. token-eco cherche à réduire les deux premiers. Les mesures visent à répondre aux questions suivantes :

- **Q1 — Fidélité et taille :** les outils renvoient-ils la bonne information, et combien de tokens économisent-ils par appel ?
- **Q2 — Coût en conditions contrôlées :** à tâche égale, une session avec token-eco coûte-t-elle moins cher, sans perte de réussite ?
- **Q3 — Adoption réelle :** dans l'usage quotidien, les agents appellent-ils vraiment ces outils ?
- **Q4 — Multi-agents :** dans un projet où un orchestrateur délègue à des agents spécialisés, qu'apportent les outils, et selon la longueur de la tâche ?

02 Système étudié

COMPOSANT	RÔLE	PRÉSENT DANS LES VARIANTES « ECO »
Serveur MCP : `read`, `search`, `edit`	lecture d'un symbole, d'une plage ou d'un plan ; recherche groupée par fichier ; remplacements groupés en un appel ; tous acceptent des lots	oui
Agent principal `token-eco:eco`	remplace l'agent par défaut ; Read/Grep/Glob/Edit natifs retirés ; consignes de regroupement (« une recherche, une lecture, une édition par étape », concision)	oui
Consigne de session	~6 lignes injectées au démarrage de la session et de chaque sous-agent	oui
Hook PreToolUse	compacte les sorties Bash bruyantes (tests, builds, git) ; refuse une fois la lecture complète d'un gros fichier en renvoyant son plan (seuil : > 300 lignes, et depuis v0.7.4 > 6 000 tokens)	oui

Une variante « eco » change donc plusieurs choses à la fois par rapport à la référence. Les bancs 3 et 6 mesurent l'effet de l'ensemble ; seul le contraste eco-before / eco-after du banc 6 isole un facteur (l'accès du relecteur aux outils).

03 Méthode commune et définitions

- **Coût :** `total_cost_usd` renvoyé par Claude Code en fin de session (sous-agents compris), c'est-à-dire le prix équivalent API. Les exécutions passent par un abonnement : ce montant mesure la consommation, il n'est pas facturé tel quel.
- **Tours :** nombre de réponses du modèle (fil principal / sous-agents), lu dans les journaux de session (`~/.claude/projects/`), avec dédoublonnage des messages répartis sur plusieurs lignes.
- **Couverture :** appels token-eco `read/search/edit` ÷ (ces appels + Read/Grep/Glob/Edit/MultiEdit natifs).
- **Lectures via Bash :** appels Bash commençant par `cat`, `head`, `tail`, `sed -n`, `grep`, `rg`, `find`, `awk` ; taille = taille du résultat.
- **Réussite :** vérification automatique propre à chaque tâche (réponse attendue, fichiers modifiés, suite de tests, test caché).
- **Référence étanche (« native ») :** outils token-eco interdits, hook désactivé, et depuis le banc 6 la copie du plugin installée sur le compte désactivée (sans quoi son agent s'appliquait aussi à la référence — défaut détecté et corrigé le 27/10).
- **Statistiques :** moyenne ± écart-type, minimum–maximum ; comparaison à la référence par un **test de permutation exact bilatéral** sur le coût. Avec 3 essais contre 3, il n'existe que 20 partitions : la plus petite valeur de p possible est **0,10**, obtenue quand les 3 valeurs d'un groupe sont toutes inférieures aux 3 de l'autre (séparation complète). Aucune correction pour comparaisons multiples n'est appliquée ; aucun résultat ne serait significatif après correction.

Niveaux de preuve utilisés dans la synthèse : Établi mesure directe, reproductible, sans ambiguïté · Fort effet net et cohérent, petit échantillon · Indicatif tendance, variance élevée ou un seul run · Non démontré.

04 Banc 1 — Qualité hors ligne (TS, JS, Python)

Objectif : Q1. Vérifier que le plan d'un fichier et la lecture d'un symbole sont fidèles, et mesurer leur taille par rapport à la lecture complète. **Périmètre :** 263 fichiers (zod et ajv en TypeScript, SDK MCP compilé en JavaScript, bibliothèque standard Python 3.15), 2 666 symboles. **Méthode :** comparaison à une vérité terrain issue de l'arbre syntaxique réel (compilateur TypeScript, module `ast` de Python) ; tokens estimés à 4 caractères par token ; référence = Read natif (2 000 lignes au plus). Gratuit, déterministe (`npm run bench` , 6,7 s).

CORPUS	FICHIERS	RAPPEL DU PLAN	PRÉCISION DU PLAN	TAILLE PLAN / READ (MOY. ; MÉD.)	SYMBOLE EX
TypeScript	88	95,7 %	92,0 %	21,6 % ; 9,9 %	
JavaScript	40	100 %	98,5 %	5,7 % ; 6,4 %	
Python	135	99,0 %	100 %	7,8 % ; 7,6 %	

Recherche compacte vs `grep -rn` (10 motifs sur zod) : décompte identique pour 10 motifs sur 10 ; 47,3 % des tokens de `grep -rn` au total, mais plus que `grep -rn` sur les motifs à 0–2 résultats (en-tête fixe), et plus que le Grep natif par défaut, qui ne renvoie que la liste des fichiers (104 tokens au total).

Conclusion : Établi les outils sont fidèles et renvoient 4 à 22 % de la taille d'une lecture complète. **Limites :** mesure de taille par appel, pas d'une session ; l'économie réelle dépend de ce que l'agent aurait lu à la place.

05 Banc 2 — Qualité multi-langages

Objectif : Q1 hors des trois langages principaux. **Périmètre :** 15 langages, jusqu'à 50 fichiers chacun, dépôts open source. **Méthode :** vérité terrain universal-ctags (`node bench/languages.ts`).

Résultats globaux : rappel du plan 96,9 %, lecture de symbole exacte 92,8 %. Taille du plan : 4,6 % (shell) à 23,5 % (PHP) de la lecture complète. Points faibles mesurés : précision du plan en C (26,6 %, minorant car ctags ignore certaines déclarations), Java et C# (≈ 68 %), lecture de symbole en Elixir (40,3 % exacte). Swift, Scala et Dart : pas de vérité terrain disponible.

Conclusion : Établi fidélité élevée pour la plupart des langages, avec des faiblesses identifiées (C, Java, C#, Elixir).

06 Banc 3 — A/B mono-agent en conditions réelles

Objectif : Q2. **Périmètre :** 5 tâches courtes (comprendre, chercher, éditer, mini-session, fonctionnalité avec tests) sur zod/src/v3 et sur le serveur du plugin ; Claude Code en mode non interactif (`claude -p`), Sonnet, 4 essais par tâche et par mode. **Méthode :** chaque essai sur une copie jetable ; échauffement non compté pour neutraliser le cache froid ; réussite vérifiée.

VERSION	TÂCHE	COÛT NATIF	COÛT ECO	ÉCART	TOURS NATIF / ECO	RÉUSSITE
v0.3	t1 comprendre	0,141 $	0,145 $	+3 %	2,8 / 3,0	4/4 · 4/4
v0.3	t2 chercher	0,135 $	0,125 $	−7 %	2,0 / 2,0	4/4 · 4/4
v0.3	t3 éditer	0,145 $	0,127 $	−12 %	5,0 / 3,0	4/4 · 4/4
v0.3	t4 mini-session	0,158 $	0,142 $	−10 %	4,5 / 4,0	4/4 · 4/4
v0.3	t5 fonctionnalité	0,187 $	0,167 $	−11 %	8,0 / 6,3	4/4 · 4/4
v0.4.1	t4 mini-session	0,147 $	0,131 $	−11 %	4,5 / 4,0	4/4 · 4/4
v0.4.1	t5 fonctionnalité	0,174 $	0,158 $	−9 %	7,5 / 4,5	4/4 · 4/4

Total v0.3 : −8 % (0,765 $ → 0,706 $) ; total v0.4.1 : −10 % (0,321 $ → 0,288 $). Coût fixe par exécution ≈ 32 000 tokens écrits en cache, qui pèse lourd sur des tâches de cette taille. Sur t4 et t4/t5 v0.4.1, toutes les sessions eco ont utilisé exactement `search → read → edit` au lieu de Grep, Read et plusieurs Edit.

Conclusion : Fort sur des tâches courtes mono-agent, −8 à −10 % de coût sans perte de réussite (40/40 en v0.3, 16/16 en v0.4.1), principalement par moins de tours. **Limites :** tâches courtes et conçues pour l'outil ; pas de test statistique ; à l'époque, la référence n'était pas protégée contre l'agent du plugin installé (défaut découvert plus tard, voir §11).

07 Banc 4 — Sessions réelles scénarisées

Objectif : Q2 sur une session complète. **Périmètre :** une session en 6 étapes sur la bibliothèque click 8.5.0 (explorer, corriger un bug, ajouter un type, renommer dans plusieurs fichiers, relecture par un sous-agent, changelog), mêmes prompts, cache chaud, 9 contrôles d'acceptation cachés. **Méthode :** coût réel lu dans les journaux (`bench/session-cost.ts`), comparé à Claude Code sans plugin. **Source :** résultats consignés dans le README ; rapports bruts non conservés.

RUN	VERSION, MODÈLE	OUTILS SEULS (MODE PAR DÉFAUT D'ALORS)	AGENT STRICT ECO
1	v0.4, Sonnet (corrigé du cache froid)	+17 %	–
2	v0.4, Sonnet	−7 %	–
3	v0.5.0, Sonnet	0 %	−5 %
4	v0.5.1, Sonnet	+22 %	+6 %
5	v0.5.3, Opus	−1 % (tokens −14 %)	−13 % (tokens −32 %, 27 tours contre 37)

Toutes les sessions ont passé 9/9. Tokens réellement traités, agent strict : −10 % (run 3) et −6 % (run 4).

Conclusion : Non démontré pas d'économie en dollars avec Sonnet sur une session complète : la variance d'une session à l'autre (±15 % et plus) dépasse l'effet. Indicatif avec Opus et l'agent strict, −13 % (un seul run). Explication observée : Sonnet seul lit déjà avec parcimonie (`grep` , `sed -n` enchaînés), alors qu'Opus seul lit des fichiers entiers.

08 Observation 5 — Usage réel sur 30 jours

Nature : étude observationnelle, pas une expérience. **Objectif :** Q3. **Périmètre :** journaux Claude Code de l'auteur sur 30 jours, plusieurs projets, hors dossiers de benchmark et temporaires. **Méthode :** lecture des journaux par le tableau de bord du plugin.

	APPELS TOKEN-ECO	APPELS NATIFS	COUVERTURE
Fil principal	21	445	4,7 %
Sous-agents	112	9 685	1,2 %

Économie estimée : environ 3,41 $ sur 4 283 $ (< 0,1 %) ; les sous-agents pesaient 44 % du coût. **Causes identifiées dans les journaux :** (a) un autre plugin imposait son propre agent principal, qui l'emportait sur `token-eco:eco` ; (b) les agents en lecture seule du projet (relecteurs, sécurité, architecture) n'avaient que `Read, Grep, Glob`, liste produite par un générateur du projet ; (c) les sous-agents lisaient via Bash : sur 9 680 appels, `cat` a renvoyé 8 Mo, `sed -n` 7 Mo, `grep` 4,8 Mo.

Après correction (même jour, autre plugin retiré, relecteurs équipés de `read` / `search`) : sur quelques heures d'un projet, un relecteur a utilisé token-eco pour 55 % de ses appels de fichiers (62 contre 50, 0 % avant) ; un agent producteur 21 % (50 contre 191).

Conclusion : Établi avant correction, les outils n'étaient presque pas utilisés : aucune économie ne pouvait en résulter, quelle que soit leur qualité. Indicatif l'adoption remonte une fois les causes levées (un projet, quelques heures).

09 Banc 6 — Adoption multi-agents

Objectif : Q3 et Q4 en conditions contrôlées : reproduire la situation observée en usage réel, mesurer l'adoption des outils par des sous-agents et son effet sur le coût, selon la longueur de la tâche. **Outil :** `server/bench/adoption.ts`, statistiques `server/bench/adoption-stats.ts`.

9.1 Protocole

- **Projet de test** (copie neuve dans `/tmp` à chaque exécution) : le serveur du plugin et ses 35 tests ; deux spécifications Markdown de 454 lignes (~3 600 tokens chacune) dont les exigences utiles sont enfouies au milieu ; un `CLAUDE.md` « tu es l'orchestrateur, tu délègues » ; un agent `reviewer` en lecture seule avec une liste `tools:` ; un agent `producer` sans liste.
- **Variantes :** *native* (plugin désactivé, référence) ; *eco-before* (plugin, agent eco, hook ; relecteur limité à `Read, Grep, Glob`, situation avant correction) ; *eco-after* (idem, relecteur doté en plus de token-eco `read` / `search`).
- **Tâches :** *t6* faire relire le code au regard d'une spécification et lister 3 écarts cachés (vérifié : les 3 constantes et valeurs citées) ; *t7* faire implémenter une exigence (vérifié : suite de tests + test caché) ; *t8, tâche longue* enchaîner 5 délégations (relecture, correction des 3 écarts, nouvelle relecture, implémentation, relecture finale ; vérifié : 3 constantes corrigées, suite complète et test caché).
- **Modèles :** orchestrateur Sonnet 5.5 ou Opus 5.5 ; les sous-agents ont tourné sur Sonnet 5.5 dans tous les cas, via le réglage `CLAUDE_CODE_SUBAGENT_MODEL` du compte, comme en usage réel.
- **Contrôles :** exécution écartée si l'agent principal n'est pas celui attendu, si la référence utilise token-eco, ou en cas d'erreur d'infrastructure (réseau, abonnement). Ordre fixe à chaque essai : native, eco-before, eco-after.
- **Exécutions :** 63 valides, **63 réussies** ; 5 écartées (perte de réseau, abonnement suspendu), relancées.

9.2 Campagne A1 — tâches courtes, v0.7.3 (36 exécutions)

ORCHESTRATEUR	TÂCHE	VARIANTE	COÛT MOYEN ± É.-T. [MIN–MAX]	VS NATIVE	P	COUVERTURE RELE
Sonnet	t6	native	0,194 ± 0,038 [0,171–0,238]	–	–	
Sonnet	t6	eco-before	0,238 ± 0,015 [0,221–0,252]	+22 %	0,20	
Sonnet	t6	eco-after	0,237 ± 0,020 [0,214–0,253]	+22 %	0,20	
Sonnet	t7	native	0,262 ± 0,010	–	–	
Sonnet	t7	eco-before	0,265 ± 0,018	+1 %	0,80	
Sonnet	t7	eco-after	0,259 ± 0,003	−1 %	0,60	
Opus	t6	native	0,391 ± 0,018	–	–	
Opus	t6	eco-before	0,406 ± 0,033	+4 %	0,60	
Opus	t6	eco-after	0,369 ± 0,021	−6 %	0,30	
Opus	t7	native	0,399 ± 0,023	–	–	
Opus	t7	eco-before	0,394 ± 0,017	−1 %	0,80	
Opus	t7	eco-after	0,386 ± 0,006	−3 %	0,50	

Couverture : relecteur pour t6, producteur pour t7 (agrégée sur les 3 essais). 3 essais par case, 36/36 réussis, 11,40 $.

9.3 Diagnostic du surcoût de t6 avec Sonnet (+22 %)

Le surcoût est identique avec et sans relecteur équipé : il ne vient donc pas de l'accès aux outils. Analyse des journaux des 36 exécutions :

- **Pas un coût fixe de contexte :** contexte du premier tour (médiane) — fil principal 27,5 K tokens en eco contre 33,0 K en native (l'agent eco retire des outils) ; sous-agent 23,2 K contre 21,1 K.
- **Pas un défaut de cache propre au plugin :** sondes « Reply OK » (5 exécutions, ~0,50 $) — en changeant de dossier, environ 20 K tokens sont réécrits en cache dans les deux variantes ; dans le même dossier, tout est relu.
- **Cause : le hook.** Décomposition (Sonnet, t6) : coût des sous-agents 0,05 $ en native contre 0,12–0,13 $ en eco. Le relecteur eco lisait la spécification ; le hook refusait (454 lignes > 300) en renvoyant le plan et en conseillant l'outil `read` que ce relecteur n'avait pas ; il relançait la lecture complète (autorisée à la seconde tentative), puis lisait 6 fichiers source en entier. Le relecteur natif lisait la spécification puis faisait une seule recherche ciblée (311 tokens).
- **Correctif v0.7.4 :** la lecture complète n'est redirigée que si le fichier dépasse aussi 6 000 tokens. Une spécification Markdown de 454 lignes courtes (~3 600 tokens) n'est plus redirigée.

9.4 Campagne A2 — t6 répété après correctif, v0.7.4, Sonnet (9 exécutions)

VARIANTE	COÛT MOYEN ± É.-T. [MIN–MAX]	VS NATIVE	P	COUVERTURE RELECTEUR	RÉUSSITE
native	0,219 ± 0,033 [0,190–0,255]	–	–	0 %	3/3
eco-before	0,194 ± 0,022 [0,169–0,210]	−11 %	0,40	0 %	3/3
eco-after	0,228 ± 0,015 [0,210–0,237]	+4 %	0,70	93 %	3/3

Le surcoût de +22 % disparaît (−11 % et +4 %, non significatifs). La moyenne de la référence varie elle-même de 13 % entre A1 et A2 (0,194 $ contre 0,219 $) : c'est l'ordre de grandeur du bruit entre campagnes.



9.5 Campagne A3 — tâche longue t8, v0.7.4 (18 exécutions)





ORCHESTRATEUR	VARIANTE	COÛT MOYEN ± É.-T. [MIN–MAX]	VS NATIVE	P	TOURS PRINCIPAL / SOUS-A
Sonnet	native	2,48 ± 0,36 [2,07–2,73]	–	–	15,3
Sonnet	eco-before	2,02 ± 0,65 [1,28–2,52]	−19 %	0,40	6,7
Sonnet	eco-after	1,30 ± 0,45 [1,00–1,82]	−48 %	**0,10**	5,3
Opus	native	4,21 ± 0,90 [3,59–5,25]	–	–	20,0
Opus	eco-before	2,89 ± 0,24 [2,74–3,16]	−31 %	**0,10**	11,0
Opus	eco-after	3,33 ± 0,65 [2,63–3,92]	−21 %	0,30	12,3

18/18 réussies, 48,64 $. p = 0,10 signifie que les trois coûts de la variante sont tous inférieurs aux trois coûts de la référence.

- **Les quatre variantes token-eco coûtent moins que la référence** (−19 à −48 %) ; deux avec séparation complète.
- **Effet de l'équipement du relecteur (eco-after contre eco-before) :** couverture des sous-agents 25 % → 64–70 % dans les deux cas ; coût −36 % avec Sonnet (p = 0,20) mais +15 % avec Opus (p = 0,40). Effet sur le coût non établi.
- **Facteur de confusion identifié :** le fil principal n'appelle aucun outil de fichier dans aucune variante (0 appel), mais fait 35 à 65 % de tours en moins sous l'agent eco (15,3 → 5,3–6,7 avec Sonnet ; 20,0 → 11,0–12,3 avec Opus). Une part du gain vient donc des consignes de l'agent principal (regrouper, être concis), indépendamment des outils de fichiers. Avec Opus, où seul l'orchestrateur est coûteux, cela peut expliquer que eco-before fasse aussi bien qu'eco-after.

10 Synthèse des conclusions et niveaux de preuve

#	CONCLUSION	NIVEAU	APPUI
1	Les outils sont fidèles (rappel du plan 96–100 % en TS/JS/Python, 97 % en moyenne sur 15 langages) et renvoient 4 à 24 % de la taille d'une lecture complète.	Établi	Bancs 1, 2
2	Sur des tâches courtes mono-agent, token-eco réduit le coût de 8 à 10 % sans perte de réussite, surtout par moins de tours.	Fort	Banc 3 (28 exécutions par mode)
3	Un outil qui n'est pas appelé n'économise rien : en usage réel, la couverture était de 1–5 % à cause d'un conflit d'agent et d'agents sans accès aux outils.	Établi	Observation 5
4	Donner les outils à un agent à liste `tools:` restreinte fait passer sa couverture de 0 % à 39–93 %.	Établi	Banc 6, toutes les cases
5	Une redirection de lecture sur un fichier moyen (≈ 3,6 K tokens) coûte plus qu'elle n'économise ; le seuil en tokens (v0.7.4) supprime ce surcoût.	Fort	9.3 (mécanisme dans les journaux), 9.4
6	Sur des tâches multi-agents courtes, pas de différence de coût mesurable (–11 % à +4 % après correctif).	Fort	A1 t7, A2
7	Sur une tâche multi-agents longue, token-eco réduit le coût (–19 à –48 % selon le modèle et la variante).	Fort pour le sens, Indicatif pour l'ampleur	A3 : 4/4 cases, 2 séparations complètes, n = 3
8	Une part du gain vient de l'agent principal (moins de tours d'orchestration), pas seulement des outils de fichiers.	Fort	A3 : tours du fil principal –35 à –65 %, 0 appel de fichier
9	Équiper le relecteur réduit le coût total de la session.	Non démontré	A3 : –36 % (Sonnet, p = 0,20), +15 % (Opus, p = 0,40)
10	token-eco réduit la facture en usage réel.	Non démontré	Banc 4 (Sonnet) ; usage réel à suivre

En une phrase : les outils sont bons et, quand ils sont réellement utilisés dans une session longue, la session coûte moins cher ; sur des tâches courtes l'effet est nul ou de l'ordre de 10 %, et en usage réel il reste à démontrer — la condition première est l'adoption.

11 Menaces à la validité

- **Statistique :** 3 essais par case (4 au banc 3) ; p minimal de 0,10 ; 14 comparaisons sans correction ; coefficients de variation de la référence de 14 à 21 % sur la tâche longue. Les ampleurs sont incertaines, seul le sens est cohérent.

- **Validité interne :** ordre des variantes fixe (native, eco-before, eco-after) et non aléatoire : un effet d'ordre ou d'heure (charge de l'API, cache) n'est pas exclu ; variantes eco = plusieurs facteurs à la fois (agent, consigne, outils, hook) ; jusqu'au 27/10, la référence pouvait hériter de l'agent du plugin installé (bancs 3 et 4 potentiellement concernés, non vérifiable a posteriori) ; une exécution écartée à la main (coupure réseau en cours d'exécution).
- **Validité de construit :** coût = prix équivalent API calculé par Claude Code ; couverture = nombre d'appels, pas tokens ; détection des lectures via Bash par motif de commande (heuristique) ; tokens estimés à 4 caractères (bancs 1–2).
- **Validité externe :** projets de test petits (≈ 35 tests, 2 spécifications) et tâches conçues par l'auteur du plugin ; un seul utilisateur ; sous-agents toujours sur Sonnet ; versions de Claude Code 2.1.29x–2.1.31x, dont le comportement évolue ; une session réelle compte des centaines à des milliers de tours, contre 37 à 95 pour la tâche longue.

12 Recommandations

- **Puissance :** pour confirmer un effet de −30 % avec la variance observée, viser au moins 8 essais par variante sur la tâche longue.
- **Ordre aléatoire** des variantes à chaque essai, et variante « agent eco seul, sans outils ni hook » pour séparer l'effet de l'agent principal de celui des outils.
- **Suivi réel** sur deux semaines avec le tableau de bord (colonne Agent, couverture, coût par tour) après les corrections.
- **Déploiement :** vérifier l'agent principal actif et donner `read` / `search` aux agents à liste `tools:` avant d'attendre une économie.

13 Annexes

Reproduction

BANC	COMMANDE (DANS `SERVER/`)	SORTIE
1	`npm run bench`	`bench/RESULTS.md`
2	`node bench/languages.ts`	`bench/LANGUAGES.md`
3	`npm run bench:ab -- --model sonnet --trials 4 [--tasks t4,t5]`	`bench/AB.md`
4	`node bench/session-cost.ts <dossiers…> [--cold]`	sortie console
6	`node bench/adoption.ts --dry` puis `--models sonnet,opus --trials 3 [--tasks t8] [--out nom] [--resume]`	`bench/ADOPTION*.md` , `*-raw.json`
Stats	`node bench/adoption-stats.ts`	console (moyenne, é.-t., p)

Coût des campagnes du banc 6

A1 : 11,40 $ (36 exécutions) · A2 : 1,93 $ (9) · A3 : 48,64 $ (18 valides) · sondes de diagnostic : ≈ 0,80 $. Montants en prix équivalent API.

Glossaire

Couverture : part des appels de fichiers passant par token-eco. **Orchestrateur** : fil principal qui délègue. **Sous-agent** : agent lancé par l'orchestrateur, avec son propre contexte. **Hook** : script exécuté par Claude Code avant chaque appel d'outil. **Séparation complète** : toutes les valeurs d'un groupe sont inférieures à toutes celles de l'autre.